

M11.5.5 — Strategic Alignment Review

Part 3 — Soft-Locked Milestone Framework

Project: FH6 Farm Tool

Review Type: Strategic Governance / Anti-Drift Framework

Purpose: Define how the project protects direction, quality, flexibility, and launch focus without becoming rigid or over-governed.

1. Purpose of This Framework

The purpose of this framework is not to create bureaucracy.

The purpose is to protect:

decision quality
project philosophy
launch focus
trust
scope discipline
small-team flexibility

The FH6 Farm Tool has moved quickly. That speed is a strength, but speed also creates risk:

stale assumptions
scope drift
feature temptation
documentation decay
over-polishing
late-stage complexity

This framework exists to prevent those risks from quietly damaging the project.

It should be used as:

a decision constitution

not:

a rigid rulebook.

The goal is:

stay disciplined without becoming inflexible.

2. Meaning of “Soft-Locked”

A soft-lock means:

this is the strongly preferred direction unless reality gives us a good reason to change it.

A soft-lock is not the same as a hard lock.

Hard lock

Used for things that should not change casually because they protect safety, trust, or validated behavior.

Example:

```
no hidden automation  
no unsafe startup behavior  
no disabling emergency stop
```

Soft lock

Used for strategic direction.

Example:

```
Discord-first launch  
Basic / Plus packaging  
restrained premium UI
```

A soft-locked decision may change, but only through explicit review.

It should not change because:

```
we got excited  
a feature sounded cool  
one user requested it  
Codex made it easy
```

It may change if:

```
new testing reveals better evidence  
user feedback strongly contradicts assumptions
```

technical reality changes
launch strategy materially improves
both founder and lead developer agree

Soft-locking protects direction while preserving adaptability.

3. Decision Categories

The project should classify future work into four categories:

Frozen
Allowed
Deferred
Experimental

This prevents all ideas from feeling equally urgent.

4. Frozen Scope

Frozen scope means:

do not casually revisit.

Frozen items are either already settled or too dangerous to destabilize without a deliberate milestone.

Currently Frozen

Safety principles

F8 emergency stop
finite execution
confirmation flags
cleanup on stop/failure
no hidden automation
no startup automation
one automation at a time

Product principles

trust-first
consumer-centered
reliability before speed
simple by default

complexity hidden until needed
premium restrained UX

Monetization principles

no artificial crippleware
Basic must remain a complete product
Plus must provide fair additional value

Engineering principles

shared systems centralized
automation modules separated
real-input guarded
dangerous behavior explicitly confirmed
startup remains safe

Frozen does not mean impossible to change.

It means:

changing it requires serious justification.

5. Allowed Scope

Allowed scope means:

aligned work that may proceed when it fits the current milestone.

Allowed work should still be prioritized carefully.

Currently Allowed

MVP hardening

command surface hardening
refusal message improvements
pre-flight clarity
operator feedback
reliability audit
edge-case handling
documentation sync

UI/product work

restrained premium desktop UI
dashboard
setup wizard
automation pages
profiles UX
logs UX
settings UX
safety/readiness UI

Packaging strategy

Basic / Plus architecture
feature boundary planning
Discord-first launch preparation
paid-product readiness

Scaling validation

Auto3 scaling validation
Auto2 scale validation
validated threshold discovery

Allowed scope is not automatic scope.

It must still respect:

current milestone
complexity budget
launch focus
trust impact

6. Deferred Scope

Deferred scope means:

important, but intentionally not now.

Deferred items are not forgotten.

They are parked with reasoning.

Currently Deferred

Website-first launch

Reason:

Discord-first is currently stronger for trust, support, and feedback.

Revisit trigger:

validated demand
support burden grows
need for broader public distribution

Advanced licensing system

Reason:

High complexity and not needed for first controlled launch.

Revisit trigger:

paid demand validated
piracy/support risk becomes meaningful
manual Discord gating becomes insufficient

Advanced updater

Reason:

Premature before distribution model stabilizes.

Revisit trigger:

user base grows
manual updates become support burden

Cloud systems / accounts

Reason:

High complexity, low immediate necessity.

Revisit trigger:

multi-device needs
license management requires it
support tooling requires it

Pricing optimization

Reason:

Exact pricing is less development-critical than packaging and trust.

Revisit trigger:

launch preparation
Basic / Plus value boundaries clearer

Broad analytics / stats layer

Reason:

Potentially valuable, but not required for trust-first launch.

Revisit trigger:

core UX stable
users request value summaries
post-launch polish

Deferred scope should be reviewed periodically, but not allowed to leak into active work without a deliberate decision.

7. Experimental Scope

Experimental scope exists because the project operates partly in a complex environment.

Some decisions cannot be known perfectly in advance.

For those, the correct approach is:

probe → sense → respond

not:

guess → hard lock → regret.

Experimental scope is the controlled outlet for:

creative ideas
complex-system learning
safe-to-fail tests
new feature investigations
strategic uncertainty

Experimental Rule

Either founder or lead developer may suggest an experiment.

But:

both must agree before it becomes active work.

This protects flexibility without allowing chaos.

Requirements for an Experiment

An experiment must be:

bounded
safe-to-fail
time-limited
clear in purpose
aligned with project philosophy
low-risk to current stability

An experiment must answer a specific question.

Bad:

let's try building a big licensing system

Good:

test whether Discord membership gating can support Basic / Plus access
without a website

Current Candidate Experiments

Discord paywall feasibility

Question:

Can Discord membership tiers support early Basic / Plus distribution cleanly?

Value:

High.

Risk:

Low.

Auto3 scaling validation

Question:

What car-count threshold remains reliable and commercially satisfying?

Candidate ranges:

10
15
25
50

Value:

High.

Risk:

Controlled if tested safely.

Plus Auto2 fast method feasibility

Question:

Can the faster Auto2 method be made reliable enough to justify Plus positioning?

Value:

High.

Risk:

Medium.

Auto4 safety prototype

Question:

Can car removal be made safe enough for Plus without threatening trust?

Value:

Medium-High.

Risk:

High.

Status:

Conditional, not mandatory.

8. Complexity Governance

The project is not anti-complexity.

The project is anti-unmanaged complexity.

Complexity may be accepted when it is:

bounded
learnable
safe-to-fail
valuable
aligned with trust
not momentum-killing

Before accepting complexity, ask:

Can we bound it?
Can we test it safely?
Can we recover if wrong?

Does the value justify the cost?
Does it support launch or distract from launch?

If yes:

complexity may be a managed investment.

If no:

defer, simplify, or reject.

This protects the project from both:

naïve ambition

and:

defensive conservatism

The goal is:

disciplined experimentation.

9. Trust as Lead Principle

Trust is the lead principle of the FH6 Farm Tool.

Not the only principle.

But the highest product landmark.

When decisions conflict, ask:

Which option increases user trust?
Which option reduces perceived sketchiness?
Which option makes the software feel safer?
Which option makes the user feel more in control?

Trust must be balanced with:

simplicity
usability
profitability

consumer expectations
technical feasibility

Trust should guide decisions, but not become blind dogma.

10. Consumer-Centered Definition of “Perfect”

The project should not define “perfect” internally as:

technically impressive
feature complete
developer satisfying

For launch, perfect means:

meets user expectations.

A feature is “perfect enough” when the user feels:

this works
this feels safe
this is clear
this is worth paying for
this does what I expected

This is especially important for:

Auto1
Auto2
Auto3
UI
setup wizard
support/onboarding

The product does not need to be flawless.

It needs to be:

expectation-meeting and trustworthy.

11. Basic / Plus Philosophy

Basic / Plus packaging remains strategically attractive.

But it must follow one hard principle:

Basic must never be intentionally bad.

The project rejects artificial crippleware.

Bad:

Basic is frustrating so users upgrade.

Good:

Basic is complete.
Plus is more powerful, faster, or more scalable.

Current Basic Direction

Basic should include the foundational product value:

Auto1
Auto2
Auto3

Provisional Basic scale direction:

Auto1: unlimited race farming
Auto2: limited batch size using robust method
Auto3: limited batch size using validated safe range

Current provisional thinking:

Auto2 Basic: around 10 cars
Auto3 Basic: around 10 cars

These numbers are not hard-locked.

They are placeholders pending validation.

Current Plus Direction

Plus should provide fair additional value.

Candidate Plus value:

```
higher Auto2 batch size
faster Auto2 method
higher Auto3 batch size
Auto4 car removal
advanced timing/profile control
expanded profile flexibility
```

Current provisional thinking:

```
Auto2 Plus: up to around 50 cars
Auto3 Plus: up to around 50 cars
```

Again:

provisional, not hard-locked.

The principle is more important than the number.

12. UI / UX Philosophy

UI is not decoration.

For this project:

UI is trust infrastructure.

A command-line public product would conflict with:

```
premium
consumer-centered
simple
safe-feeling
trust-first
```

Therefore:

launch-quality UI is mandatory before public paid launch.

The UI ambition is:

restrained premium

Meaning:

above-average polish
calm
spacious
simple
guided
safe-feeling
consumer-centered
professional

Not:

flashy
RGB
gamer aesthetic
overdesigned
animation-heavy
luxury for its own sake

UI should follow the uploaded UI strategy documents, especially:

dashboard as command center
setup wizard as trust builder
automation pages as risk-sensitive flows
profiles as presets
settings as system preferences
logs as friendly + diagnostic layers

Future UI-specific amendments may refine this further after deeper review.

13. Auto3 Scaling Philosophy

The current 4-car limit reflects current validation history.

It should not become an artificial launch ceiling.

The correct launch principle is:

validated threshold, not arbitrary restriction.

Auto3 should scale only as far as reliability supports.

The question is not:

Can we technically allow 50?

The question is:

Can we confidently support 50?

Scaling must be validated.

Possible validation ladder:

4 cars
10 cars
15 cars
25 cars
50 cars

The final launch threshold should be selected based on:

reliability
user expectations
support burden
Basic / Plus packaging
trust impact

14. Auto4 Philosophy

Auto4 is not mandatory for launch.

Auto4 is also not rejected.

Current position:

conditional pre-launch candidate

Auto4 may be considered if it can be made:

strongly guarded
clearly explained

testable
reversible in confidence but not in action
Plus-appropriate
safe enough to protect trust

However, Auto4 must not delay launch unnecessarily.

Question to ask:

Is early user trust and feedback more valuable than car removal?

Current likely answer:

yes.

Therefore:

Auto4 remains a strong Plus candidate, but not a launch blocker.

15. Final Polish as Hard Launch Gate

Final polish is not optional.

M13 must exist as a hard launch gate.

Final polish includes:

UI polish
documentation cleanup
operator clarity
safety wording
onboarding refinement
trust pass
engineering polish
small reliability fixes
technical debt cleanup

Final polish may include real code changes.

That is expected.

But those code changes must be:

targeted
bounded
trust-improving
launch-relevant

not:

new major features
scope expansion
last-minute ambition drift

Launch does not happen when code is merely done.

Launch happens when the product is:

finished enough to deserve user trust.

16. Major Scope Freeze Review

Before final launch preparation, the project should run a deliberate:

Major Scope Freeze Review

Purpose:

Determine whether the product is feature-complete for launch.

This review asks:

Is anything genuinely missing?
Is any missing feature a launch blocker?
Is any proposed feature actually just ambition?
Does adding more increase trust or delay learning?

If the review concludes the product is feature-complete:

no major new features after that point.

Literally:

nothing major gets added

After freeze, allowed work is limited to:

```
bug fixes
safety fixes
UI polish
documentation updates
launch preparation
small trust-improving refinements
```

This protects launch momentum.

17. Documentation Freshness Framework

Documents are valuable only when their freshness is understood.

The project should classify docs by freshness expectation.

Tier 1 — Living Operational Docs

Fast-changing.

Examples:

```
PROJECT_STATE.md
command_index.md
operator runbooks
```

Update when:

```
milestone completes
command behavior changes
current status changes
validated boundaries change
```

These are living snapshots.

Tier 2 — Validation History Docs

Mostly stable.

Examples:

reliability reports
validation reports
test reports

Update when:

validation reality changes
old statements become misleading
new validation supersedes old assumptions

Do not rewrite history unnecessarily.

Clarify and append when possible.

Tier 3 — Strategic Doctrine Docs

Long-lived.

Examples:

strategic alignment review
soft-locked milestones
product philosophy
launch threshold
deferred scope register

Update only when:

strategy changes materially
major milestone alters direction
founder/lead developer explicitly revise doctrine

These should not churn constantly.

18. Founder / Lead Developer Operating Model

The project currently benefits from a strong small-team model:

founder/client provides product judgment, priorities, and strategic instincts
lead developer/architect provides structure, technical reasoning, and challenge

Codex executes scoped implementation work
GitHub repository acts as source of truth

This model should continue.

However:

The founder/developer pair must preserve:

honest disagreement
high standards
explicit decisions
scope discipline
willingness to revise

Either side may challenge assumptions.

Neither side should preserve an old conclusion just because it was already written.

Better reasoning wins.

19. What Should Not Be Forgotten

Several items are intentionally not active right now.

They should remain visible.

Auto4
advanced Auto2
Auto3 scaling
Basic / Plus gating
Discord paywall
website later
pricing refinement
UI guideline review
final polish
documentation freshness
licensing approach
support workflow

These should not drown into implicit memory.

They should remain parked, named, and periodically reviewed.

20. Final Soft-Locked Framework Summary

The project should operate under this framework:

Frozen

Safety, trust, anti-crippleware, startup safety, emergency stop, simplicity-first principles.

Allowed

MVP hardening, UI, command hardening, reliability work, validated scaling, launch preparation.

Deferred

Website, advanced licensing, cloud systems, updater, advanced analytics, pricing optimization.

Experimental

Safe-to-fail investigations approved by both founder and lead developer.

Lead principle

Trust.

Supporting principles

Simplicity, usability, profitability, consumer expectations, technical feasibility.

Launch standard

Paid-product standard Controlled Premium MVP.

UI standard

Restrained premium, trust-first desktop experience.

Complexity philosophy

Avoid unmanaged complexity, not complexity itself.

Scope freeze

Formal review before final launch preparation.

Final polish

Non-negotiable launch gate.

Final Recommendation

This framework should be treated as the project's:

anti-drift constitution

It should prevent:

scope chaos
cheap monetization
feature bloat
trust erosion
overengineering
late-stage ambition drift
documentation decay

while preserving:

small-team flexibility
creative experimentation
fast learning
strategic adaptation
high-quality execution

The framework is strict enough to protect the project.

But flexible enough to let it grow intelligently.